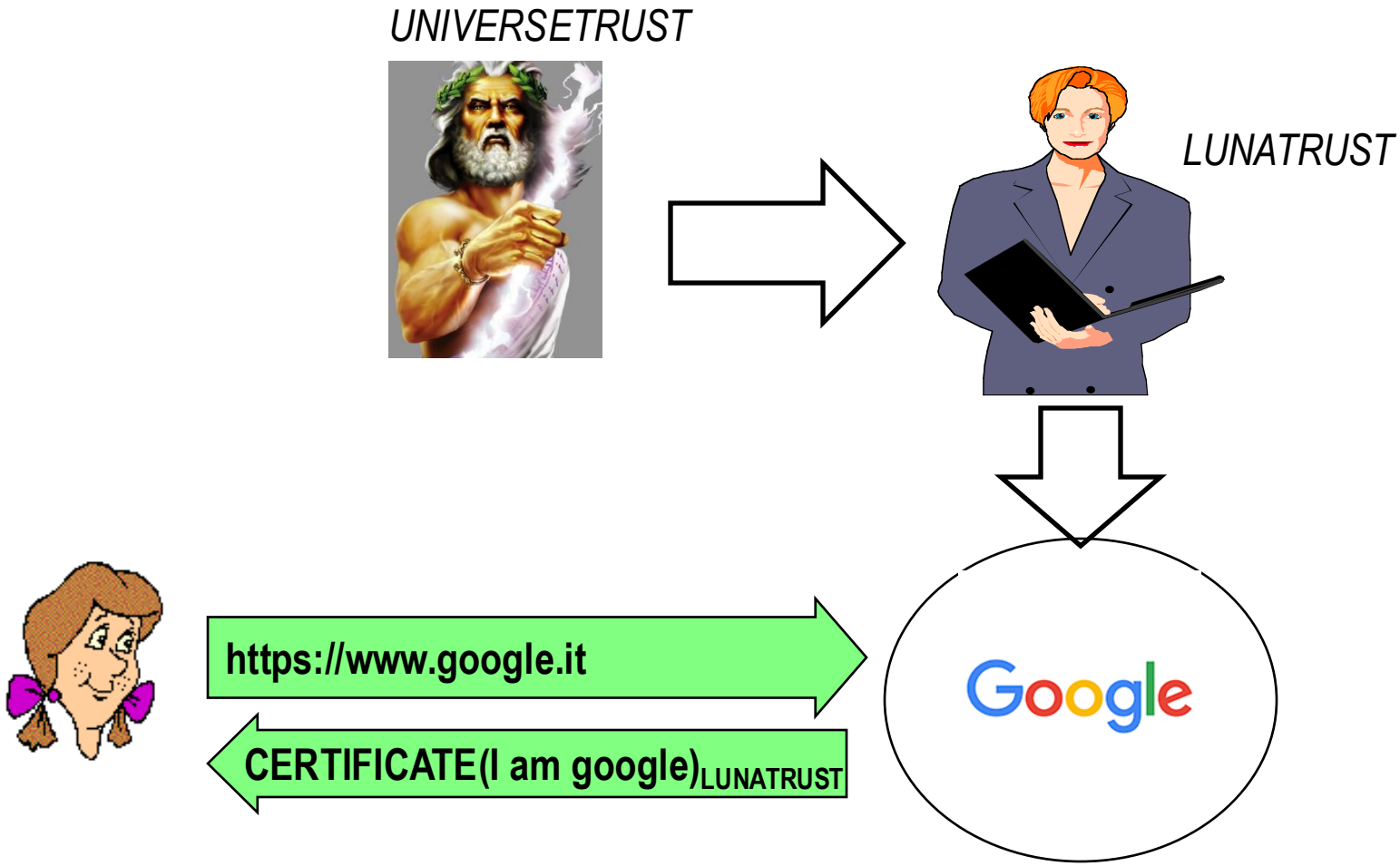
The failure of PKI
→ Certificate Transparency

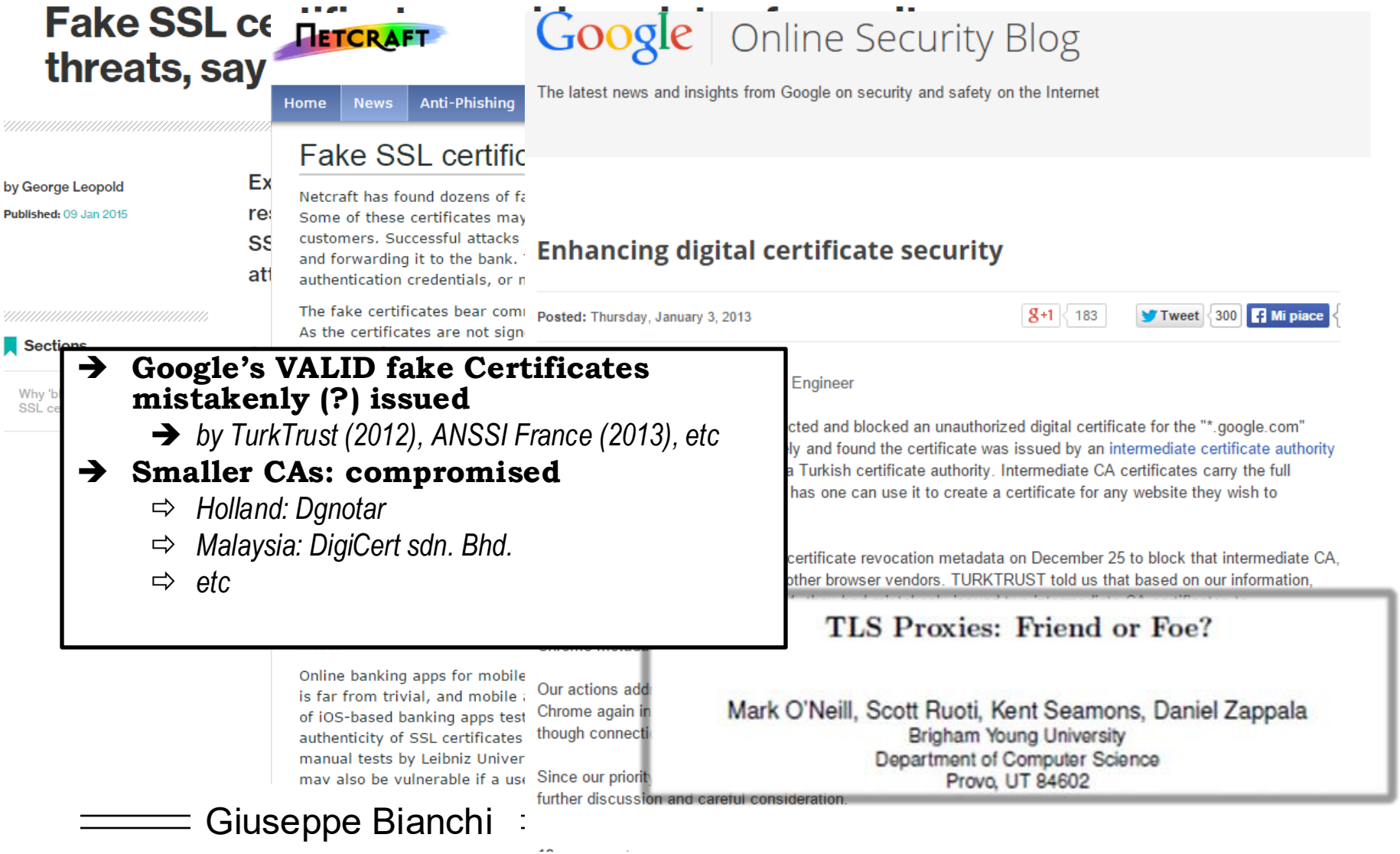
(with insights on hash based data structures, specifically Merkle trees)

a VERY SERIOUS real world problem: fake certificates

Web security pillar: Certificate Authorities ARE trusted!



Fact: trusted CA assumption at stake



- ➔ **Google's VALID fake Certificates mistakenly (?) issued**
  - ➔ *by TurkTrust (2012), ANSSI France (2013), etc*
- ➔ **Smaller CAs: compromised**
  - ⇨ *Holland: Dgnotar*
  - ⇨ *Malaysia: DigiCert sdn. Bhd.*
  - ⇨ *etc*

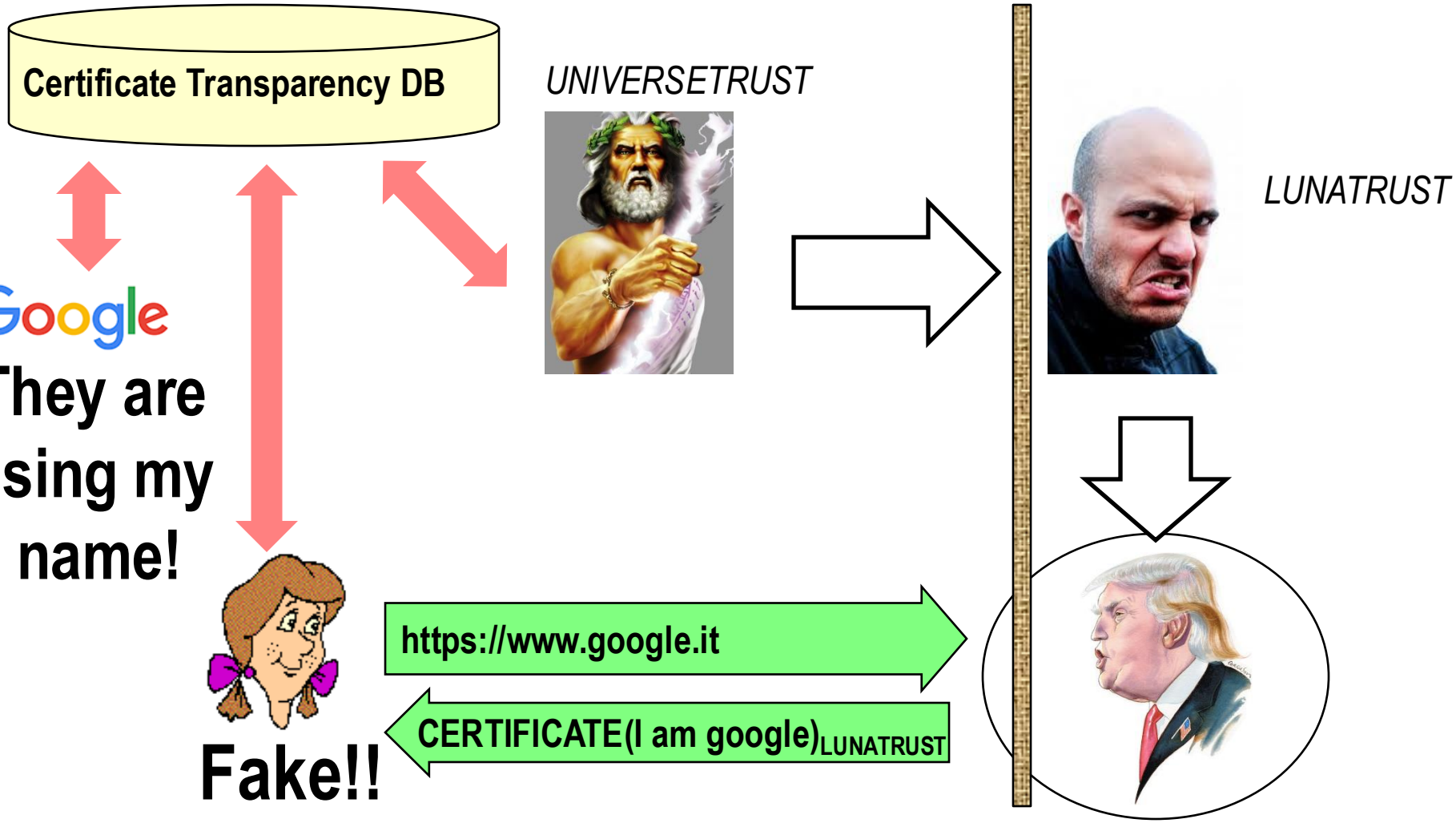
A serious problem

- **Fake VALID certificates**
  - *Variuous small/moderate CA «releasing» valid certificates for major sites*
    - Asked by governments? Mistakenly… lost?
    - Many cases:
      google fake certificates issued by
      - Turkey's Turktrust, December 2012
      - ANSSI in France on December 2013
      - etc
- **Conmpromised CAs**
  - *Holland: DGnotar*
  - *Malaysia: DigiCert sdn. Bhd.*
  - *etc*

With powerful threats (governments), and many players which can «make mistakes» the security of the PKI model is getting weaker and weaker

How to cope with malicious CAs?

Idea: gigantic worldwide DB which anyone can check!

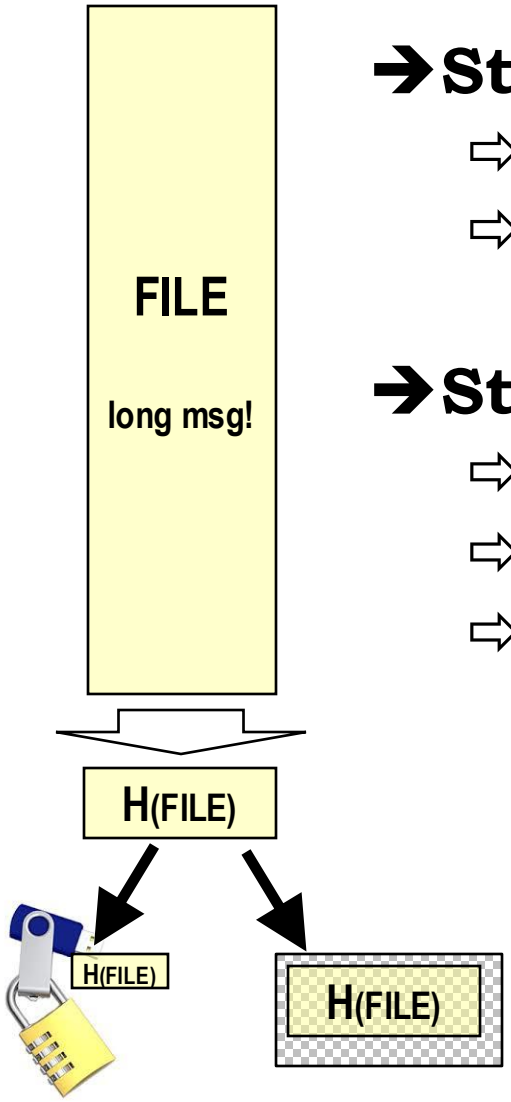


How to implement such gigantic Database of certificates?

A parethesis: Merkle Trees

How to secure a file

(when you don't have a large secure storage available)



➔Step 1: make "fingerprint" of the file

- ⇨ *Via a cryptographic hash function*
- ⇨ *large file → small fingerprint!*

➔Step 2: "secure" the fingerprint!

- ⇨ *By copying it in a secure storage*
- ⇨ *… or…*
- ⇨ *By digitally signing the hash fingerprint*

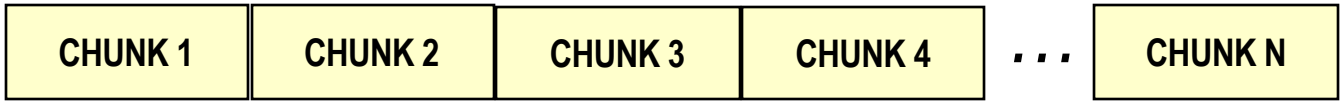
What about «chunked» messages?



➔**Peer-to-peer delivery**

⇨*Message divided into independent chunks*

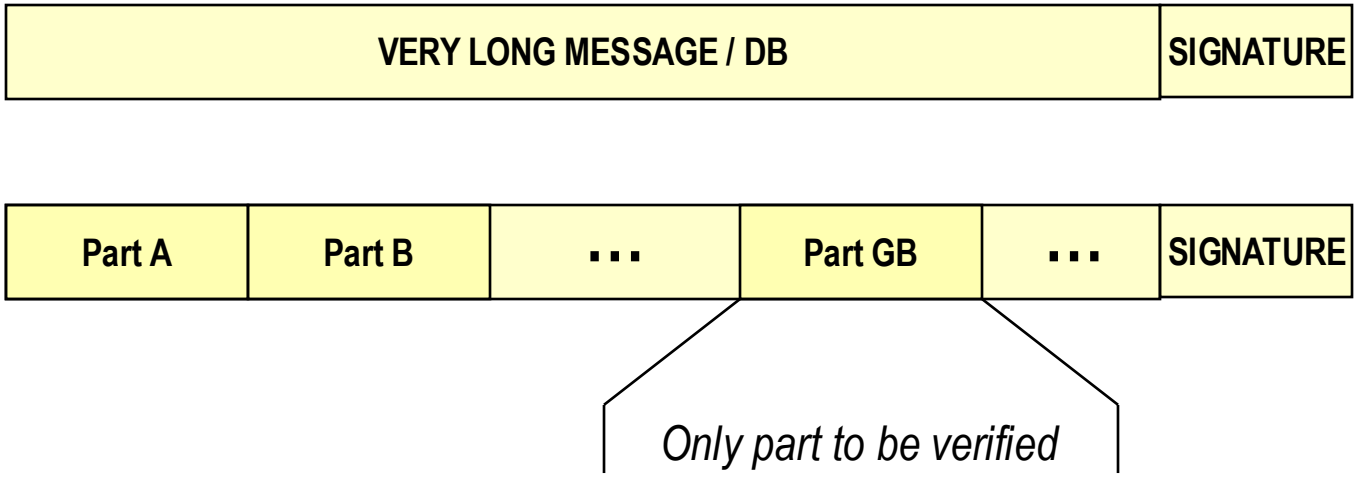
⇨*Frequently received out of order, and from distinct peers*



➔**Signature verification: needs to wait until COMPLETE message reconstruction**

⇨*But what about fake injected chunks?*

What about «partial» verification?



Signature verification:
need to retrieve the whole database??

Per-chunk signatures?



➔ **OK but…**

➔ **Computational overhead**

⇨ *Signature = anymmetric crypto = o(1000+) x hash computation cost*

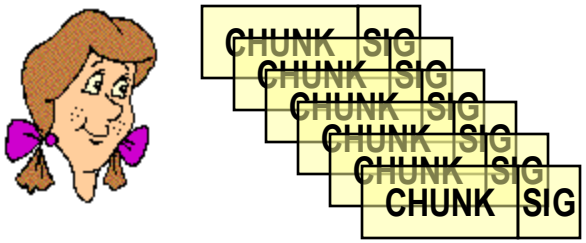
➔ **Storage overhead**

⇨ *RSA2048 = 256 bytes*

⇨ *Bitcoin's ECDSA256 = 2 x 256 bit = 64 bytes*

→still quite a lot f you need MANY of them (small chunks)

➔ **No more integrity check for whole file (strip-type attacks)**



Per-chunk signatures?



➔ **OK but…**

➔ **Computational overhead**

⇨ *Signature = anymmetric crypto = o(1000+) x hash computation cost*

➔ **Storage overhead**

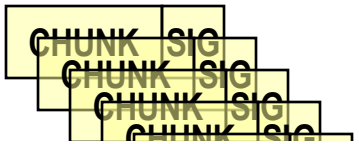
⇨ *RSA2048 = 256 bytes*

⇨ *Bitcoin's ECDSA256 = 2 x 256 bit = 64 bytes*

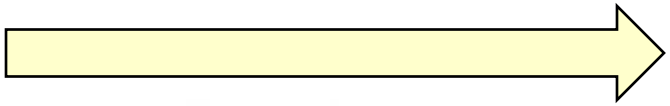
→still quite a lot f you need MANY of them (small chunks)

➔ **No more integrity check for whole file (strip-type attacks)**





Can we find a better solution?

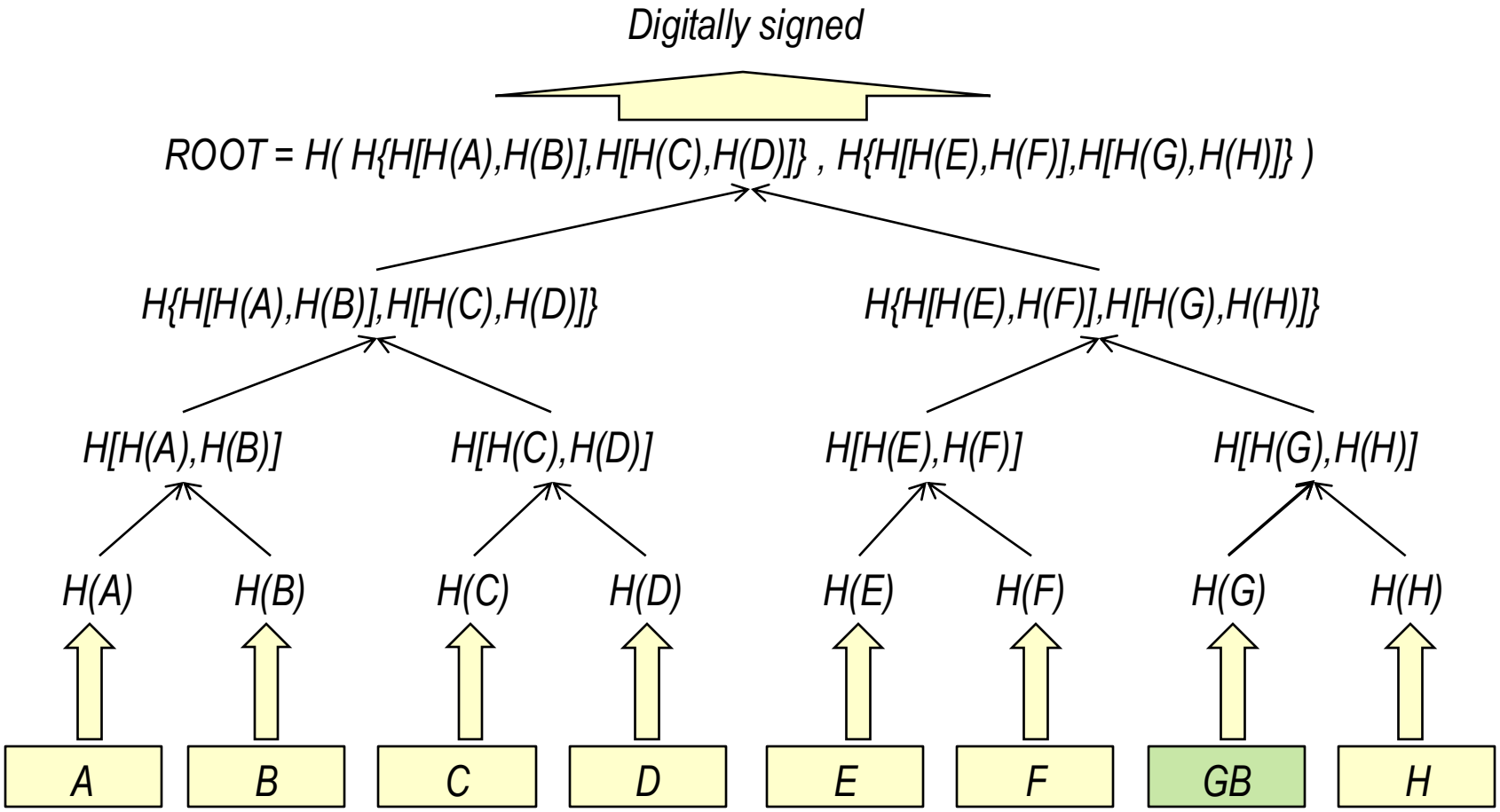






Ops… did I receive all?

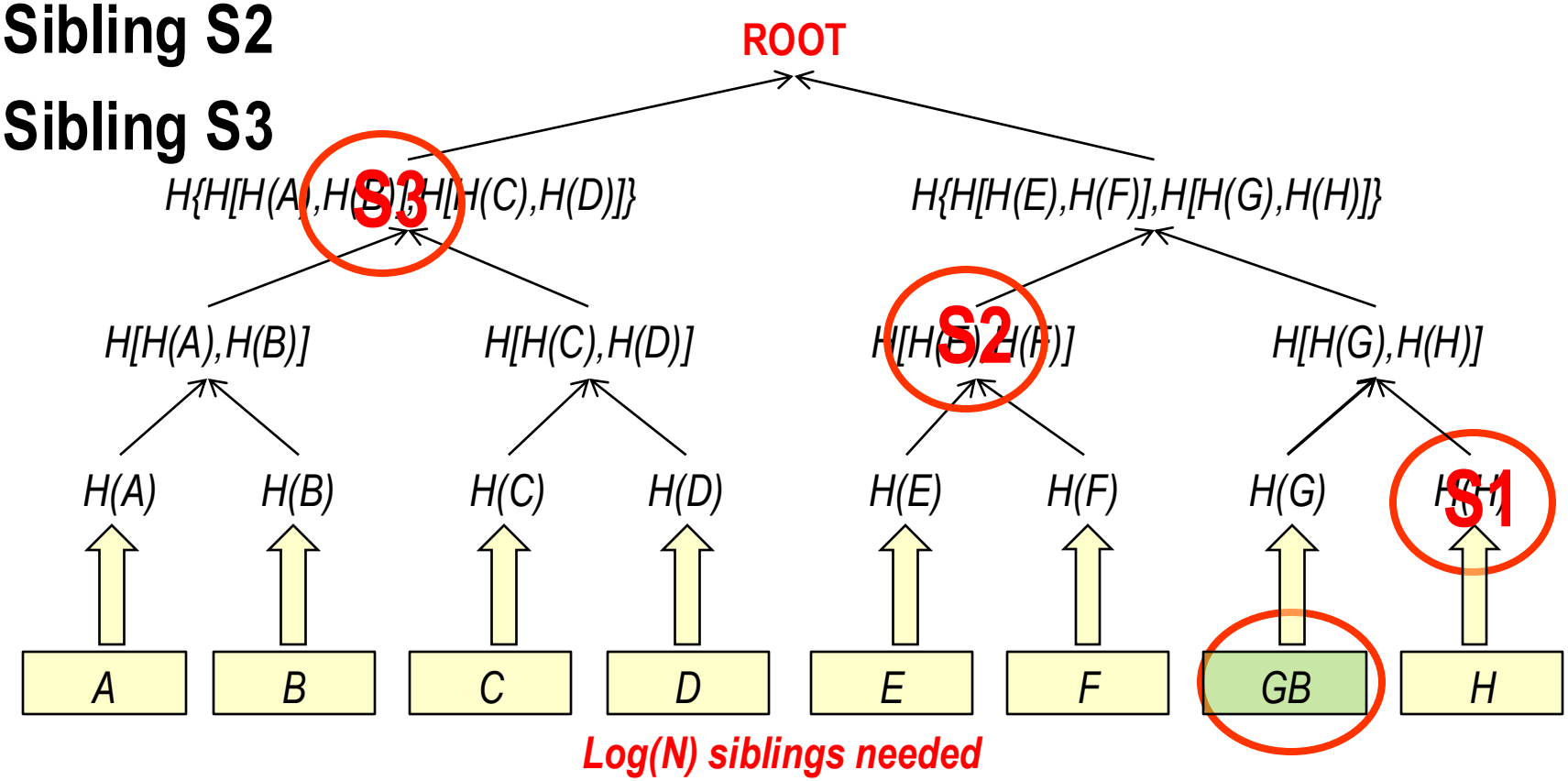
You can have them both! Merkle's trees (1979)



Single CERT verification: fast with "siblings"!

Give me:

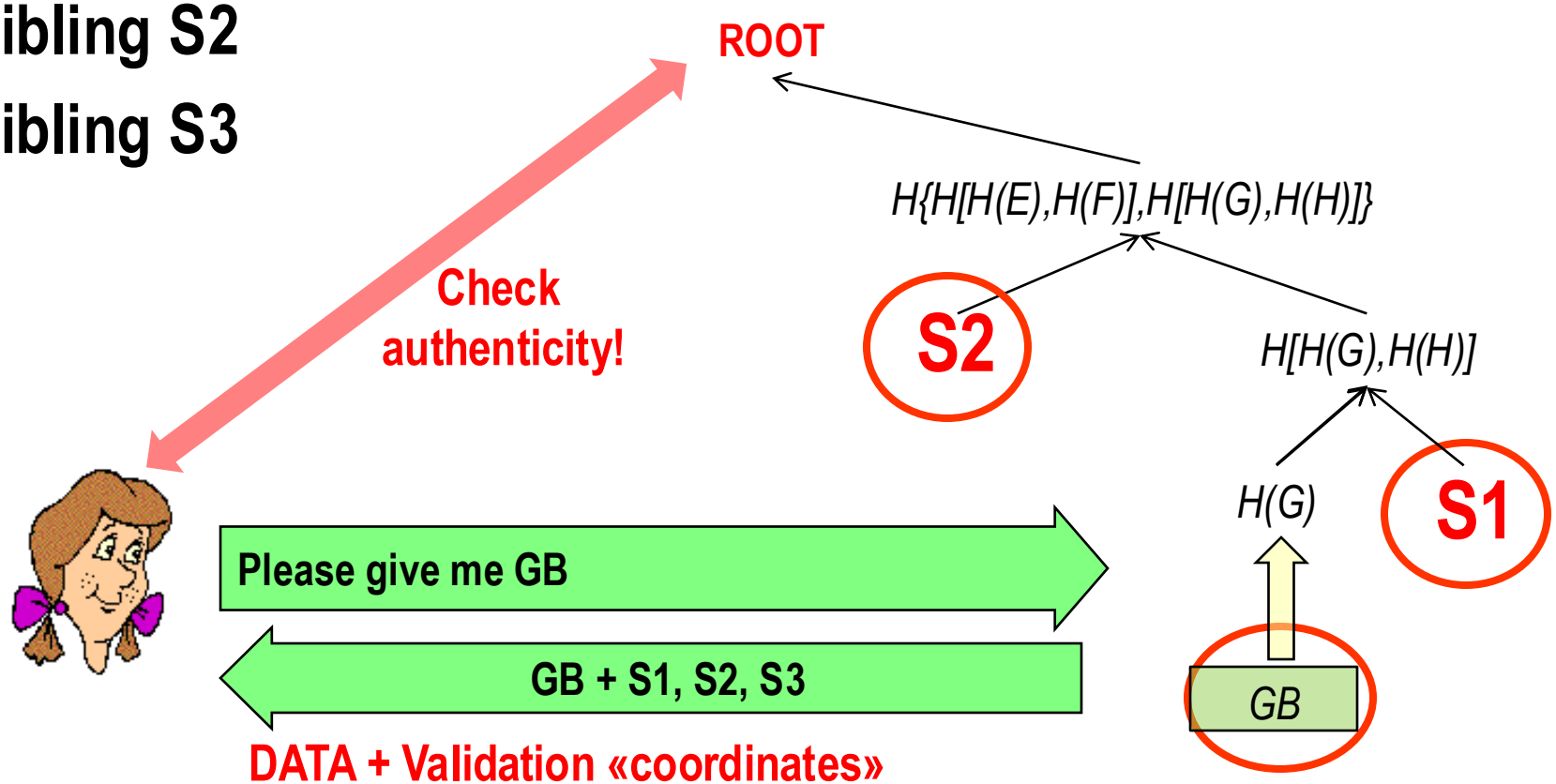
- Cert GB
- Sibling S1
- Sibling S2
- Sibling S3



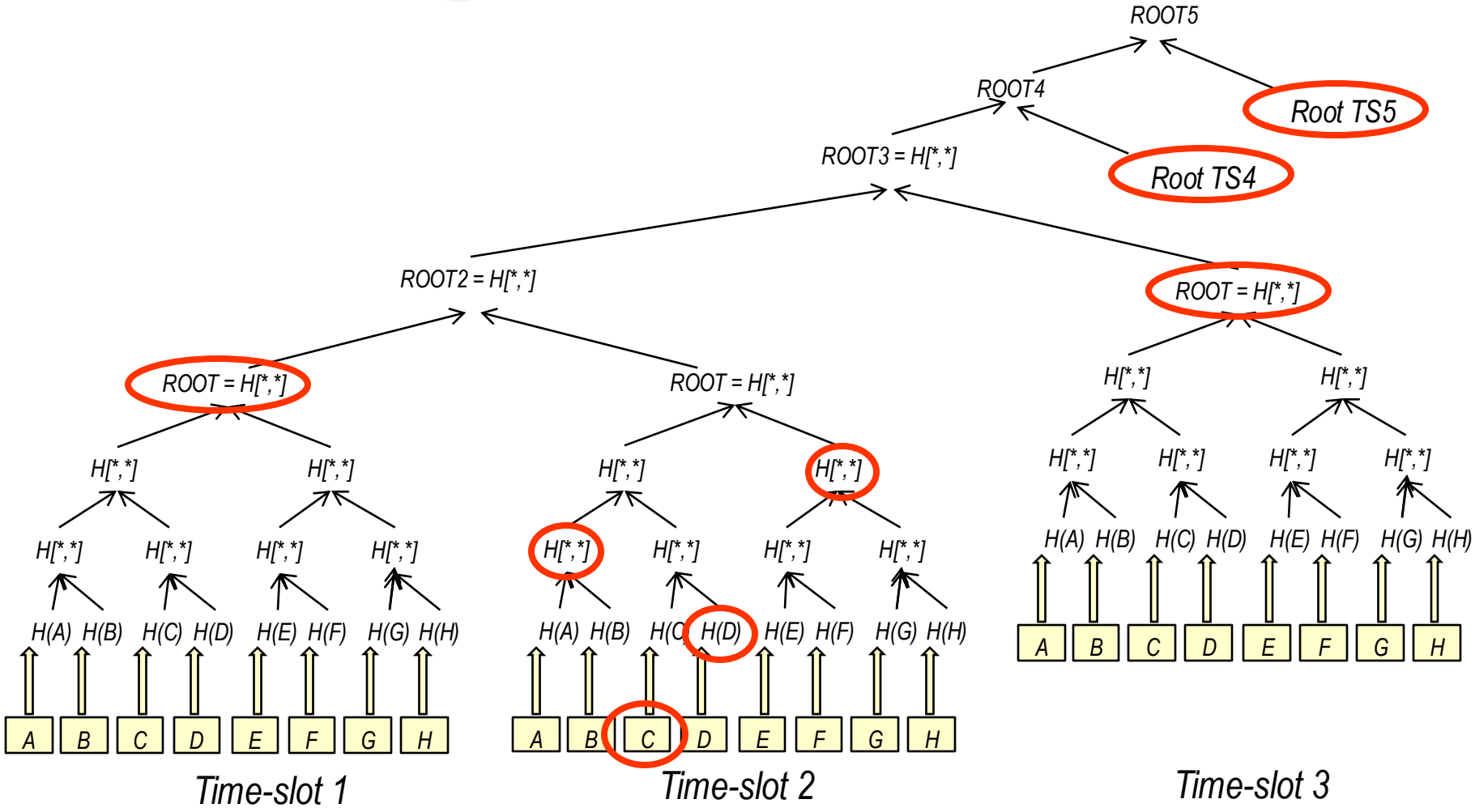
Single CERT verification: fast with "siblings"!

Give me:

- Cert GB
- Sibling S1
- Sibling S2
- Sibling S3

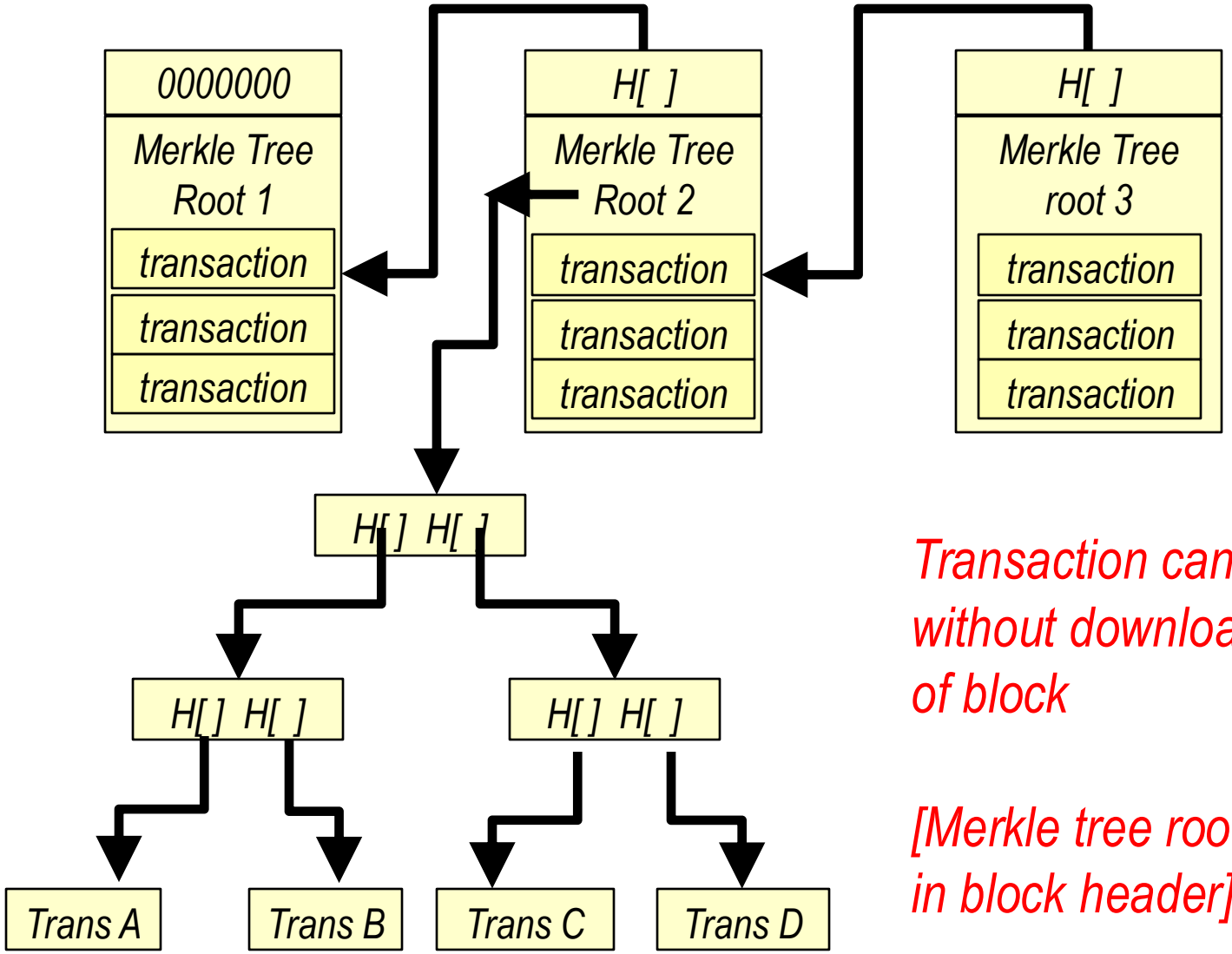


Extending merkle's trees w. time



VALIDATION: Siblings for «your» tree + root ot previous timeslot + roots ot next timeslots

Merkle trees and blockchains



Transaction can be verified without downloading content of block

[Merkle tree root stored in block header]

Merkle Trees: Applications

➔**Efficient storage in blockchains**

- ⇨*Bitcoin, Ethereum, etc*
- ⇨*Used in many parts of many blockchains!*

➔**Google's Certificate transparency**

➔**File Chunk validation**

➔**And many more**

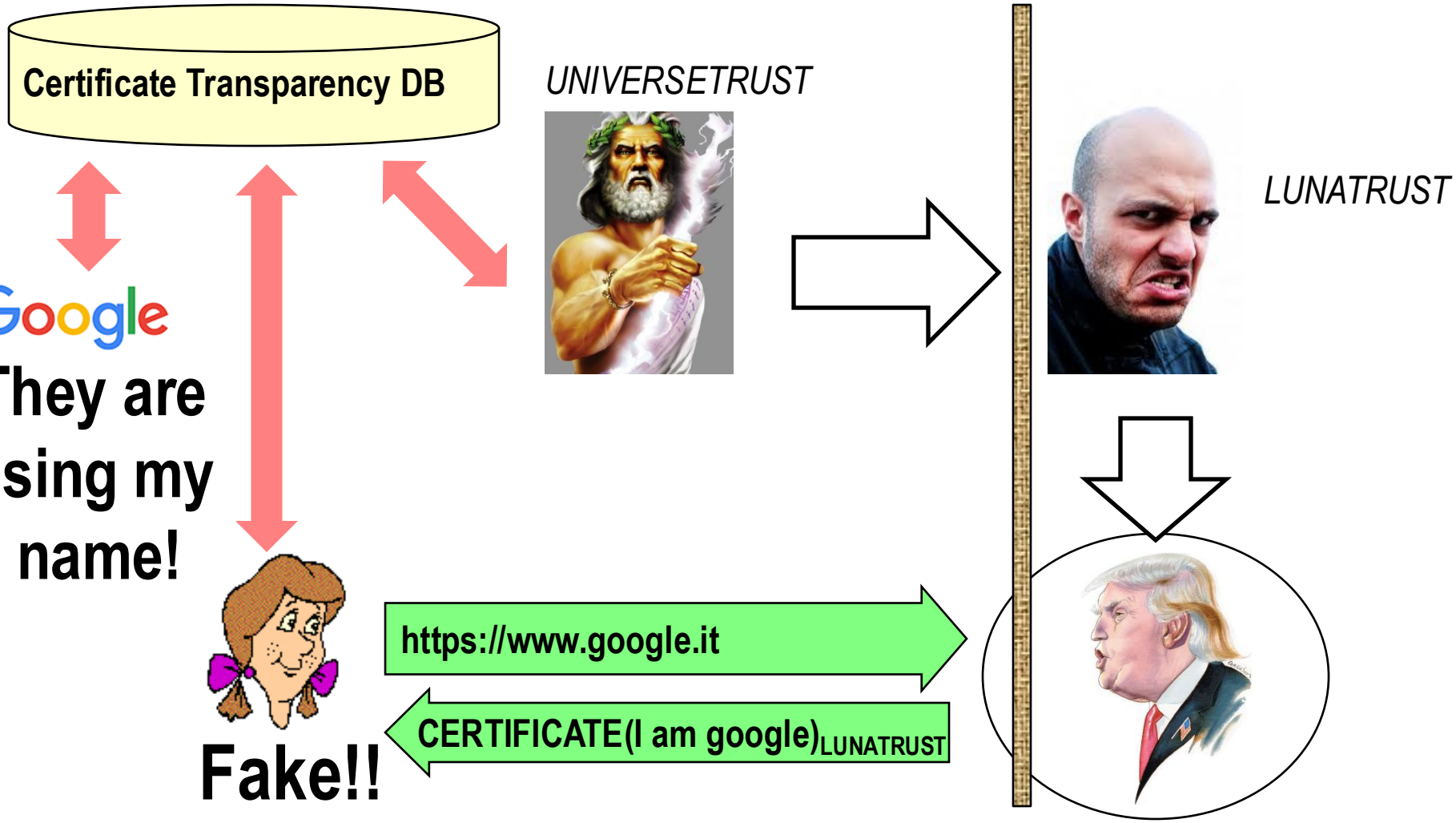
- ⇨*One-time signatures*
- ⇨*Node authentication*
- ⇨*Memory & storage authentication*
- ⇨*Signature spreading*
- ⇨*…. Etc etc …*

Merkle Trees' application:

Certificate Transparency

How to cope with malicious CAs?

Idea: gigantic worldwide DB which anyone can check!

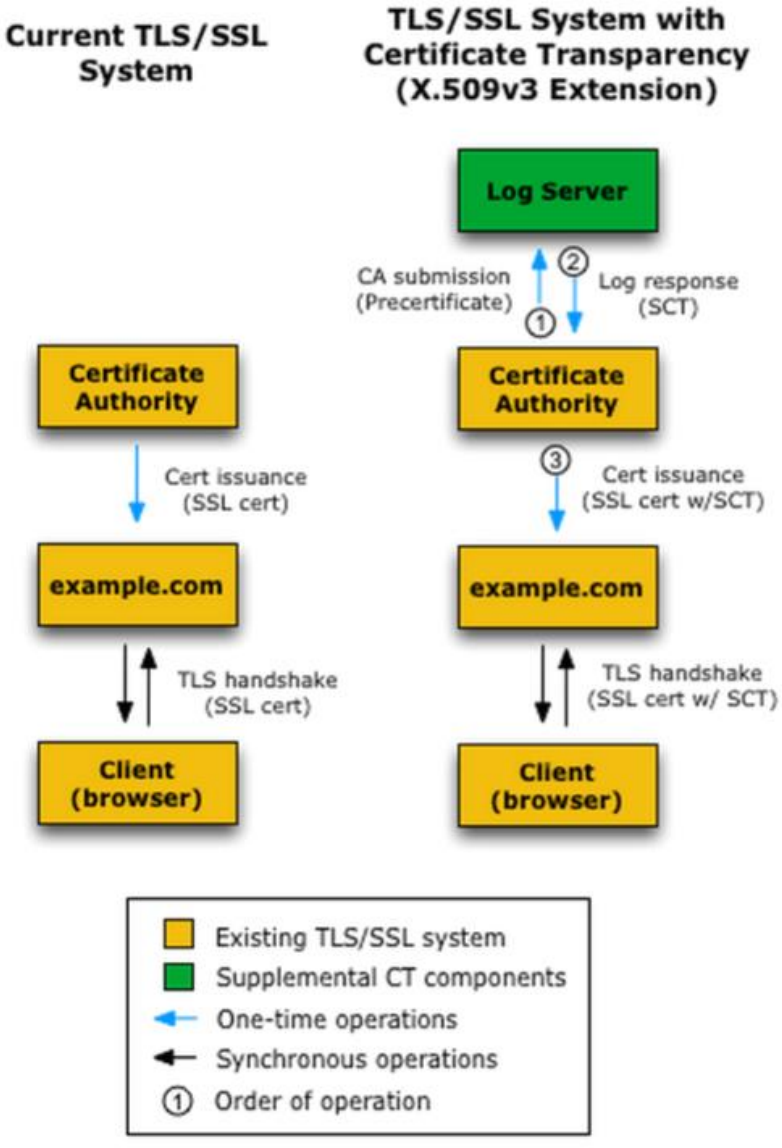


Why certificate transparency?

- **Make it almost impossible for a CA to issue a SSL certificate for a domain without the certificate being visible to the owner of that domain.**
- **Idea: put all certificates in a gigantic public list, that everyone can access and consult**
- **Further idea: make sure that we all see the "same" list!!**
  - *New threat: "split world": the public list you see is different (and crafted!) from what the rest of the world sees*

Certificate Transparency in a nutshell

- **When issuing certificate: add it to the log server**
  - *Returns a Signed Certificate Timestamp (SCT)*
- **During TLS handshake:**
  - *Check certificate validity (as usual)*
  - *Follow up with an* **additional** *Extended Validation (EV) by verifying that the certificate is in the public list*



Certificate Transparency

- **Google's initiative**
  - *started 2013*
- **First stated in experimental RFC 6962**
  - *July 2013*
- **Endorsed and supported by IETF WG «trans»**
  - *Goal: standardize CT*
  - *RFC 6962bis released in December 2014*
- **Integrated in major browsers**
  - *Chrome 1+, IE7+, Firefox 3+, Safari 3.2+, Opera 9.5+*
  - *Although not yet consistently used by all web sites*
- **Supported by major sites**
  - *Paypal, CertSign, etc*
- **Multiple pilot logs**
  - *Google, Cloudflare, DigiCert, etc*
  - *https://certificate.transparency.dev/logs/*
- **https://crt.sh/**

CT: how it works?

- **Uses Merkle trees!**
  - *Very scalable, log(n) verification complexity*
- **A special type (sometimes called chron tree)**
  - *«append only» Merkle tree*
  - *Timestamped entries*
    - Cannot retroactively insert a certificate

Following figures: source http://www.certificate-transparency.org/log-proofs-work

CT: how it works?

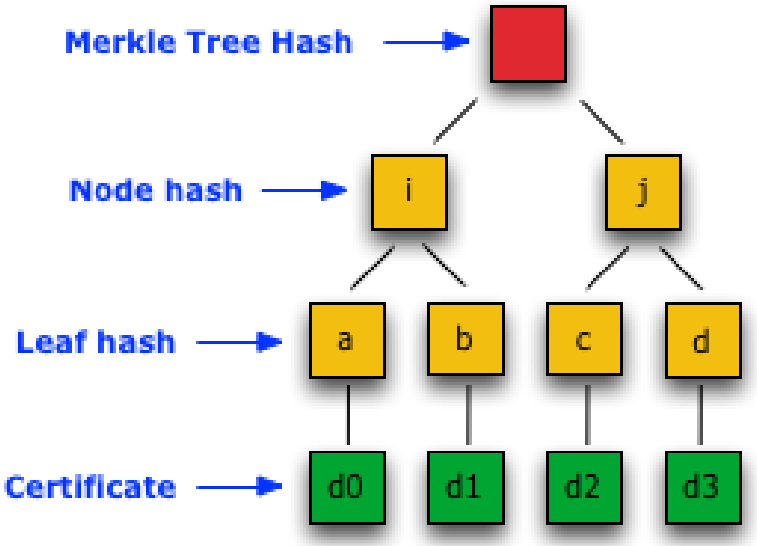


Figure 1

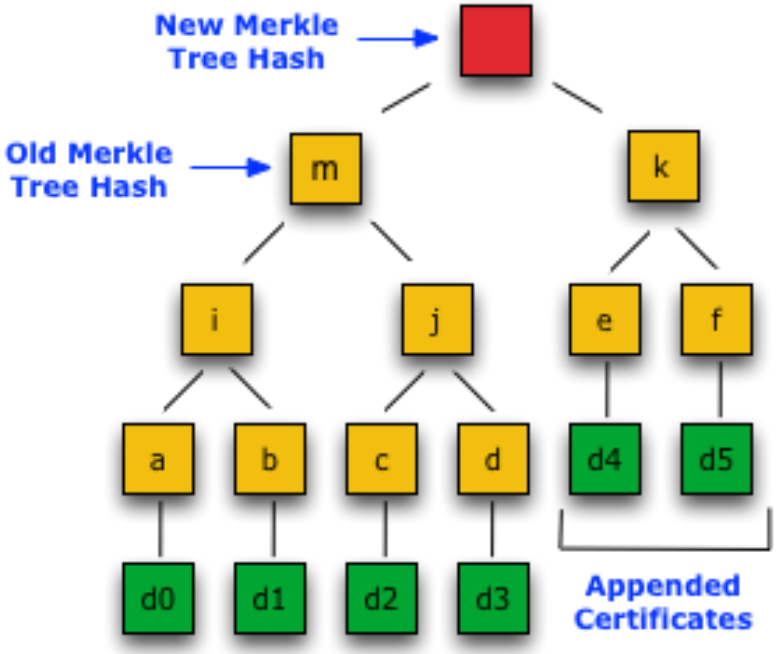


Figure 2

Every X hours, tree updated (currently X=24h)

Merkle Consistency proof

Verify that two versions of a log are consistent (i.e., the later includes everything of the earlier).
Consistency proof → no certificates have been modified or back-dated and inserted into the log.
Proof = need only root of the subtrees – includes past "root"

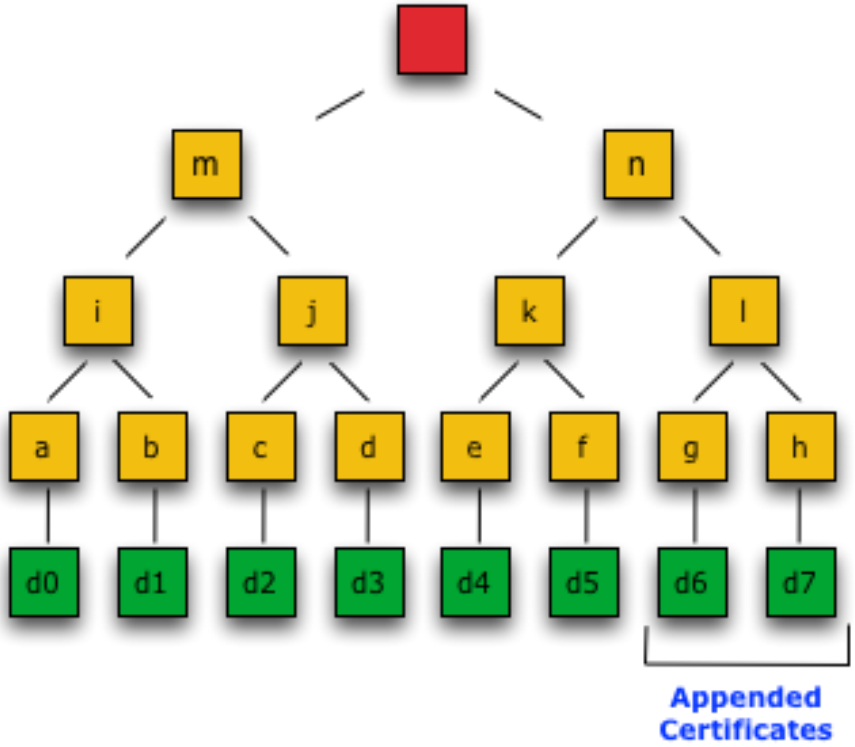


Figure 3

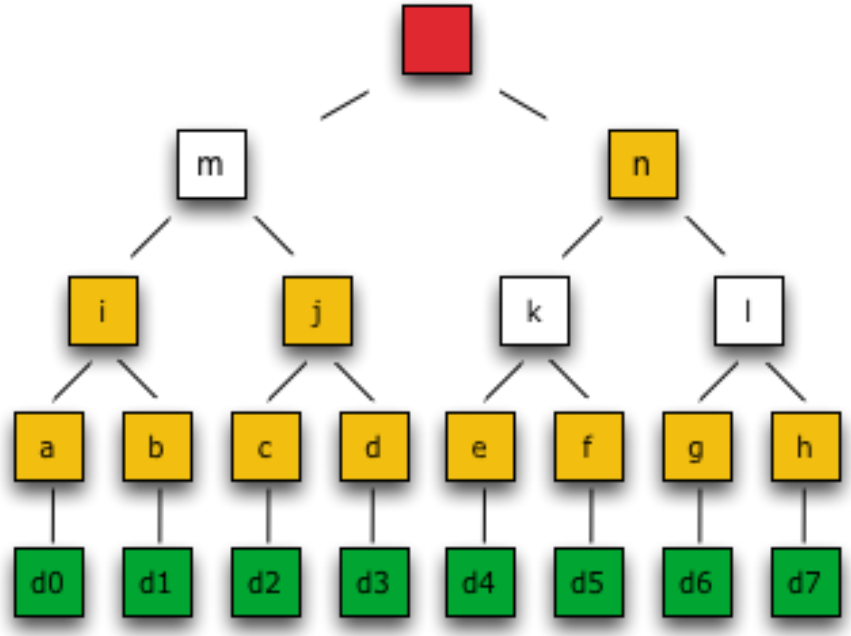


Figure 4

Merkle Audit Proof

- **Usual: via certificate + siblings**
- **Siblings obtained from log**
  - *Issue: online operation, visible to all…*
  - *Remember who are the threats today…*
- **Alternative: use «gossiping» protocols**
  - *Work in progress @ 2015!*
  - *No time for (very nice) details…*

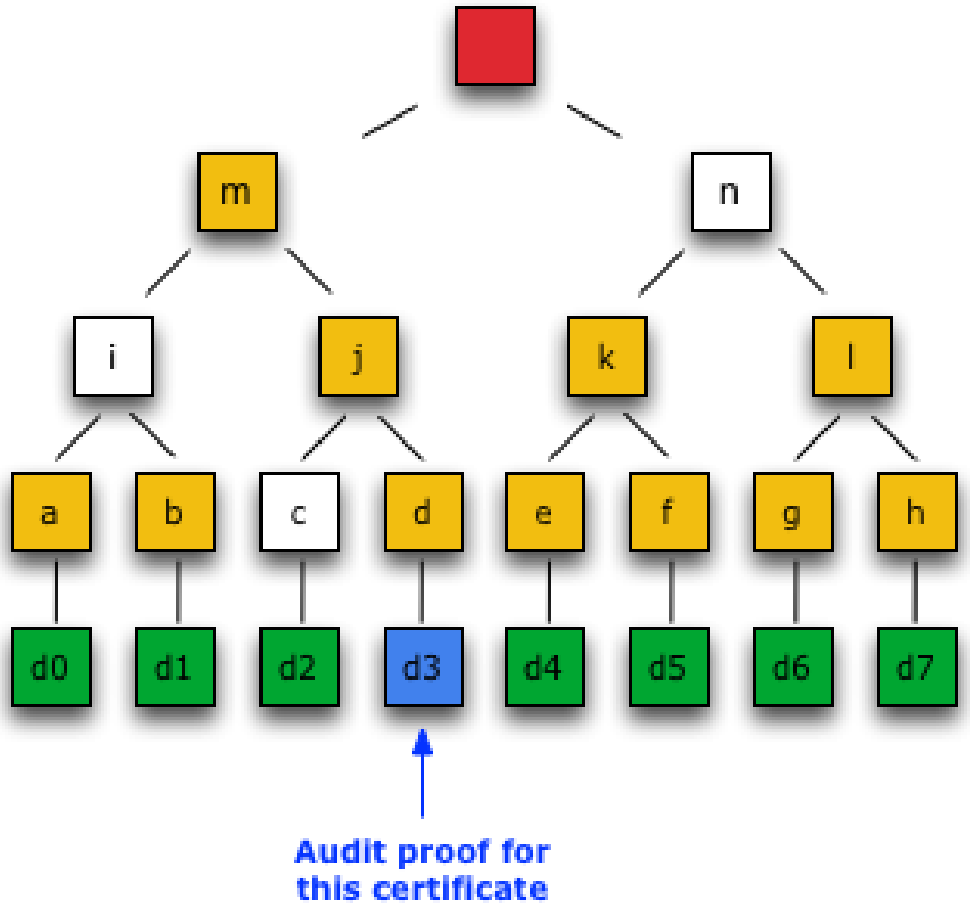
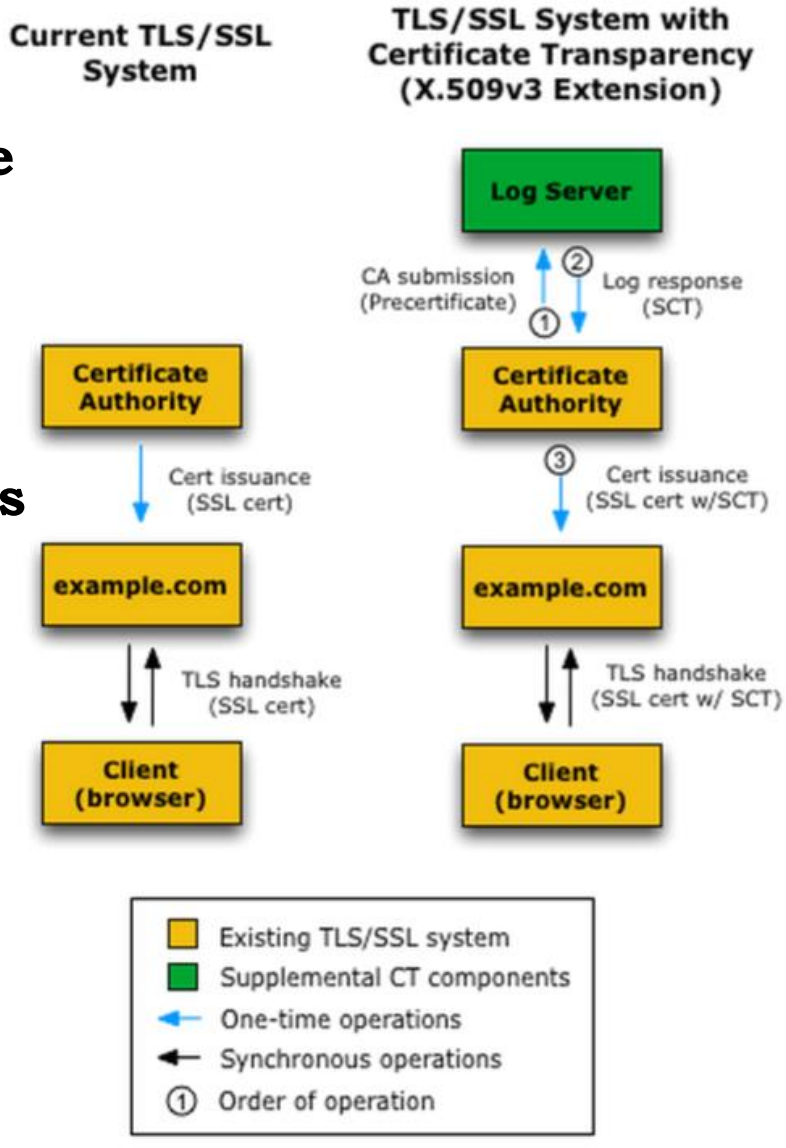


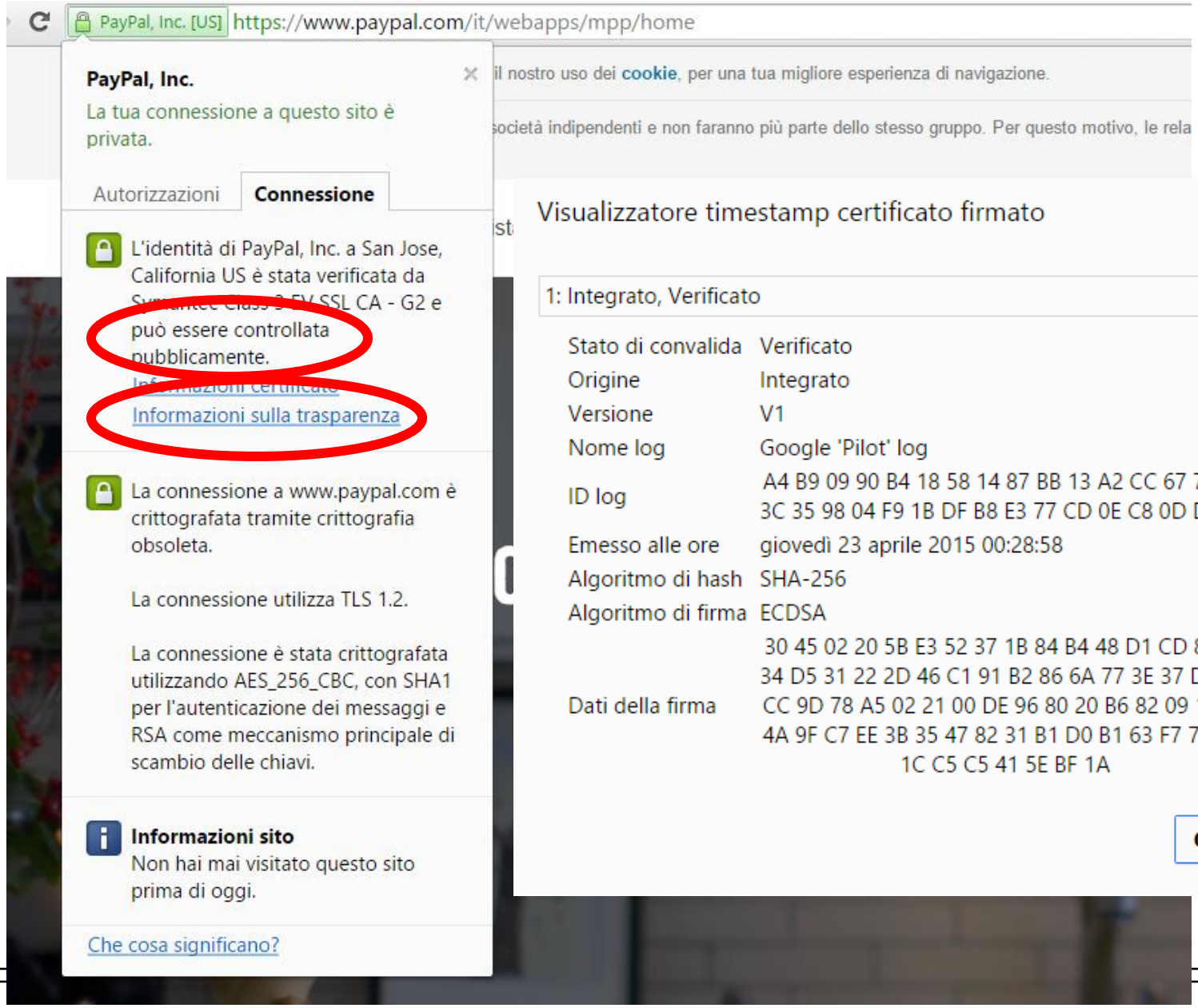
Figure 5

Certificate Transparency in the real world

- ➔ **Lauched in july 2013 by Google**
- ➔ **experimental IETF RFC 6962**
  - ➔ *IETF WG «trans»*
  - ➔ *Specified processes and protocols*
- ➔ **Integrated in all major browsers**
- ➔ **Supported by major sites**
  - ➔ *Paypal, CertSign, etc*
- ➔ **Multiple pilot logs**
  - ➔ *Google, Cloudflare, DigiCert, etc*
  - ➔ *https://certificate.transparency.dev/logs/*



EV in the real world



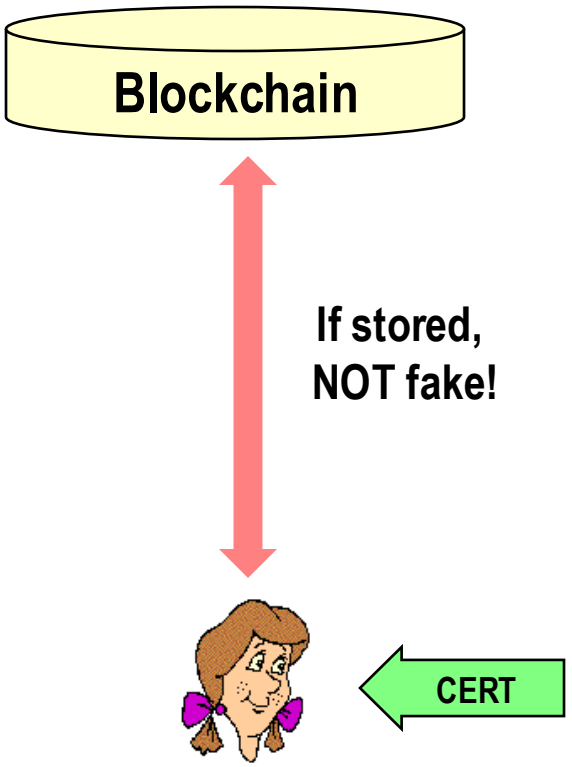
Common misconception: Cert Trans is a Blockchain?

➔ **It definitely looks like**
- ⇨ *Identical (!) architecture as Bitcoin's ledger*

➔ **It could be distributed as well**
- ⇨ *Instead of different logs, make it a same «consensus» log*

➔ **But it is NOT a Blockchain**
- ⇨ *It does NOT necessarily contain only VALID certificates!*
  - → As a Blockchain mandates; also Google being crystal clear on this!

➔



Cert Trans → is a Blockchain?

- **→ It definitely looks like**
  - ⇨ *Identical (!) architecture as Bitcoin's ledger*
- **→ It could be distributed as well**
  - ⇨ *Instead of different logs, make it a same «consensus» log*
- **→ But it is NOT a Blockchain**
  - ⇨ *It does NOT necessarily contain only VALID certificates!*
    - → As a Blockchain mandates; also Google being crystal clear on this!
- **→ Security comes from transparency**

